CODEGEN RAG CAPSTONE

Evaluating and Enhancing Code Language Models with Retrieval-Augmented Generation

AIML PGCP Capstone · Group 39, Batch 26 · TalentSprint / Accenture

Mahin Nandipa · **Abhinaya Thavishi** · **Ashu Bagul**

Mentors: Pawan Baswani, Nayan Anand · Supervisor: Prof. Anil Neelkanti

July 2026

MOTIVATION

Problem Statement & Objectives

The Problem

- Small open code models are cheap to run but lag proprietary LLMs on most benchmarks.
- codegen-350M-multi has never seen Rust — a real gap in language coverage.
- Unclear how much retrieval augmentation actually helps a small model vs. a frontier LLM.

Objectives

- Benchmark the baseline across 5 SE tasks with correlation-validated metrics.
- Extend the model to Rust via LoRA and full fine-tuning.
- Build a hybrid dense + AST RAG pipeline and measure its lift.
- Ship it: FastAPI + Streamlit + Docker + tests.

FOUNDATION

Literature Review

CodeGen

Nijkamp et al., 2022

Base model family & multi-turn synthesis framing

Codex / pass@k

Chen et al., 2021

HumanEval functional-correctness methodology

CodeBERT

Feng et al., 2020

Encoder-based similarity, informs retrieval design

Spider

Yu et al., 2018

Cross-domain text-to-SQL, unseen-schema generalization

BIRD

Li et al., 2023

Real-world, messy production-database SQL benchmark

CoDocBench

Pai et al., 2024

Code/documentation/commit-message paired dataset

RAG

Lewis et al., 2020

Parametric generator + non-parametric retrieval memory

ReACC

Lu et al., 2021

Sparse + dense retrieval for code completion

CodeBLEU

Ren et al., 2020

AST- and dataflow-aware structural similarity metric

CodeBERTScore

Zhou et al., 2023

Soft, embedding-based semantic similarity metric

Full corrected citations and a 12th, unverifiable reference are documented in the companion Research Paper Summaries.

APPROACH

Four Model Tiers, One Fair Comparison

Small LM Baseline

codegen-350M-multi, untouched

1

Fine-Tuned Rust Model

LoRA + full fine-tune extension

2

Upper-Bound LLM

Claude Sonnet 4, GPT-5 fallback

3

RAG-Augmented LLM

Dense + AST retrieval, fused

4

Every task is scored across all four tiers so gains from fine-tuning and retrieval are measured, not assumed.

DESIGN

System Architecture

Data Layer

CoDocBench, Spider, BIRD, CodeParrot, Rust corpus

→

Model Layer

codegen-350M-multi + LoRA / full fine-tune

→

Task Modules

Synthesis, docs, commits, translation, SQL

→

RAG Pipeline

FAISS + AST index, RRF fusion

→

Deployment

FastAPI + Streamlit, Docker Compose

Every layer shares one typed Settings object — four checkpoints, one coherent system, not four demos.

FOUNDATION

Data Pipeline & Datasets

Dataset	What it is	Used for
CoDocBench	Code / documentation / commit-message triples	Documentation & commit-message tasks
Spider	200 databases, cross-domain text-to-SQL	SQL generation (unseen-schema generalization)
BIRD	Large, real-world, messy production databases	SQL generation (harder, realistic difficulty)
CodeParrot	General code corpus	RAG corpus + general fine-tuning source
Rust subset	Filtered Rust code from CodeParrot	Language-extension fine-tuning task

OUTCOMES IN STAGES · STAGE 1

Foundation, Baseline Eval & Rust LoRA

- Colab/local environment bootstrap: dependency install, Drive mount, GPU detection, seeded reproducibility.
- Downloads CoDocBench + a Rust corpus subset.
- Evaluates the untouched base model on 4 tasks: program synthesis, documentation gen, commit messages, code translation.
- Produces the project's first fine-tuned checkpoint: a LoRA adapter on a 500-sample Rust subset.

1

STAGE 1 OUTCOME

checkpoints/
rust_lora_subset
(first Rust checkpoint)

OUTCOMES IN STAGES · STAGE 2

SQL Generation & Full Fine-Tuning

- Downloads full Spider + BIRD datasets with their SQLite database archives.
- Schema introspection & injection ahead of each question.
- Evaluates with exact match AND true execution accuracy (queries actually run against SQLite).
- Replaces the LoRA subset run with a full-corpus Rust fine-tune — same trainer, TensorBoard/W&B logging, disk-safe checkpoint pruning.

2

STAGE 2 OUTCOME

checkpoints/
rust_full
(full fine-tune)

OUTCOMES IN STAGES · STAGE 3

Retrieval-Augmented Generation Pipeline

- Embeds a 2,000+ sample corpus (CodeParrot + CoDocBench) with the same embedding method as task modules.
- Builds a FAISS dense index and a separate AST structural index.
- Sweeps Top-K (1/3/5/10 + dynamic threshold) × strategy (dense / AST / hybrid RRF fusion).
- Runs the full four-tier comparison and produces the comparison table + charts.

3

STAGE 3 OUTCOME

Best retrieval config + 4-tier comparison table

OUTCOMES IN STAGES · STAGE 4

Deployment

- Exposes every task + the RAG pipeline via FastAPI: /generate /document /sql /rag /translate.
- Four-tab Streamlit demo UI as a thin HTTP client.
- Containerized (separate API/UI Dockerfiles) and orchestrated with docker-compose.
- This phase's engineering additions: checkpoint-aware model routing + real tree-sitter CodeBLEU (next slides).

4

STAGE 4 OUTCOME

Live FastAPI + Streamlit demo, Dockerized

ENGINEERING DEEP DIVE

Checkpoint-Aware Model Routing

Before

Every endpoint served the base model — a trained, sitting-on-disk Rust checkpoint was never actually used in deployment.

After

_best_checkpoint_dir() resolves the best checkpoint per language: prefers training_state.json's recorded "best", falls back to highest step, then flat layout.

The Real Bug

CheckpointManager.save() nests every save one level deeper (checkpoint-NNNNNN/) than the run directory — the first lookup checked the wrong depth and silently never found the already-trained checkpoints.

Verified live in the Streamlit demo:
"rust: fine-tuned checkpoint active (checkpoint-002200)"

ENGINEERING DEEP DIVE

Genuine AST-Aware CodeBLEU

CodeBLEU = weighted n-gram + AST match + dataflow match. Without a Python grammar, the AST/dataflow terms silently fall back to a plain sacrebleu approximation.

Added

tree-sitter-python == 0.21.0

Verified against

tree-sitter == 0.22.3
codebleu == 0.7.0 (pinned)

Avoided

0.23.0+ raises TypeError
(ABI mismatch, newer Language() binding)

Result: CodeBLEU now reflects real structural + dataflow similarity, not a text-only proxy.

QUALITY

Testing Strategy

147

tests passing
(0 failing)

< 5 seconds
no GPU · no network · no model weights

How: fakes at every heavy call site

- FakeCodeGenModel / FakeAPIModel stand in wherever generation or embedding is needed.
- FastAPI dependency_overrides substitutes fakes for model, RAG indexes, LLM client in endpoint tests.
- Checkpoint-routing tests build real nested checkpoint-NNNNNN + training_state.json fixtures on tmp_path, covering every resolution branch (full-vs-LoRA, missing, empty, flat-layout fallback).

FINAL OUTCOMES

Baseline Comparison — Small LM Tier

From the project's own comparison_table.csv (Checkpoint 3 evaluation run).

Task	N	Exact Match	CodeBLEU	BERTScore F1	Exec. Acc.
SQL Generation (Spider)	200	—	—	—	0.07
SQL Generation (BIRD)	200	—	—	—	0.00
Program Synthesis	100	0.00	0.1046	0.8762	—
Commit Message Gen	100	0.00	0.0002	0.8167	—
Code Translation	29	0.00	0.0424	0.8194	—
Documentation Gen	100	0.00	0.0217	—	—

Fine-tuned Rust, upper-bound LLM, and RAG-augmented tiers follow the same table once those Colab runs complete (see Limitations).

FINAL OUTCOMES

What the Numbers Say

1 Program synthesis is the base model's strength

Highest CodeBLEU (0.1046) and BERTScore F1 (0.8762) — the task it was most directly pretrained for.

2 Exact match ≈ 0 is expected, not a failure

Open-ended generation rarely matches references verbatim — CodeBLEU and BERTScore exist precisely to capture partial correctness.

3 CodeBLEU and BERTScore tell different stories

Commit messages score near-zero CodeBLEU (0.0002) but a strong 0.8167 BERTScore F1 — short, natural-language-heavy text has little AST/dataflow structure to match.

4 BIRD is harder than Spider, as expected

0.00 vs 0.07 execution accuracy at the small-LM tier — consistent with BIRD's real-world database complexity.

LIVE DEMONSTRATION

FastAPI Backend + Streamlit Demo

Program Synthesis	Documentation Gen	SQL Generation	RAG Comparison

rust: fine-tuned checkpoint active (checkpoint-002200)

Health check confirms deployment is automatically serving the best available fine-tuned Rust checkpoint — not silently falling back to the base model.

IMPACT

Applicability in the Real World

Cost-Efficient Developer Tooling

Self-hosted small models avoid per-token API costs — this project's 4-tier method tells you exactly how much quality you give up.

Underrepresented Languages

The Rust fine-tuning recipe generalizes to legacy (COBOL, Fortran) or internal DSLs most LLMs never saw in pretraining.

Natural-Language Database Access

Text-to-SQL lets non-technical stakeholders query data in plain English — evaluated the way that matters: execution accuracy.

RAG Grounded in Proprietary Code

The hybrid dense + AST retrieval pipeline grounds generation in an org's own codebase, cutting hallucinated APIs without retraining.

Developer Productivity Automation

Documentation and commit-message generation automate two of the most commonly skipped engineering chores.

A Reusable Deployment Blueprint

FastAPI + Streamlit + Docker + checkpoint-aware routing is a directly reusable pattern for any internal model-serving tool.

TRANSPARENCY

Limitations & Honest Scope Note

Built to be fully correct and runnable — not yet executed end-to-end against a GPU, live Claude/GPT-5 APIs, or the full multi-gigabyte datasets in the development sandbox used to write and test the code.

147/147 tests passing

Every module unit/integration tested with fakes standing in for real model + network calls.

Baseline results are real

Section 6.2's numbers are the first true run — small-LM tiers only so far.

Pending: fine-tuned / LLM / RAG tiers

Await the full Colab GPU run with live API keys to populate the rest of the comparison table.

One citation unverified

"AST retrieval (Jiang et al., 2023)" could not be matched to a specific paper — flagged, not fabricated.

CONCLUSION & FUTURE WORK

A Complete, Tested Pipeline — With a Clear Path Forward

Delivered

- 5-task baseline evaluation with correlation-validated metrics
- Rust language extension via LoRA + full fine-tune
- Hybrid dense + AST RAG pipeline with RRF fusion
- Checkpoint-aware, containerized, tested deployment (147 tests)

Next

- Full GPU run with live API keys — complete the comparison table
- Reproduce & resolve the PEFT "missing adapter keys" warning
- Resolve the unverified AST-retrieval citation
- Extend the fine-tuning recipe to more languages

Thank You

Questions & Discussion

Mahin Nandipa · Abhinaya Thavishi · Ashu Bagul

Group 39, Batch 26 — AIML PGCP Capstone